

第三章 公钥密码学

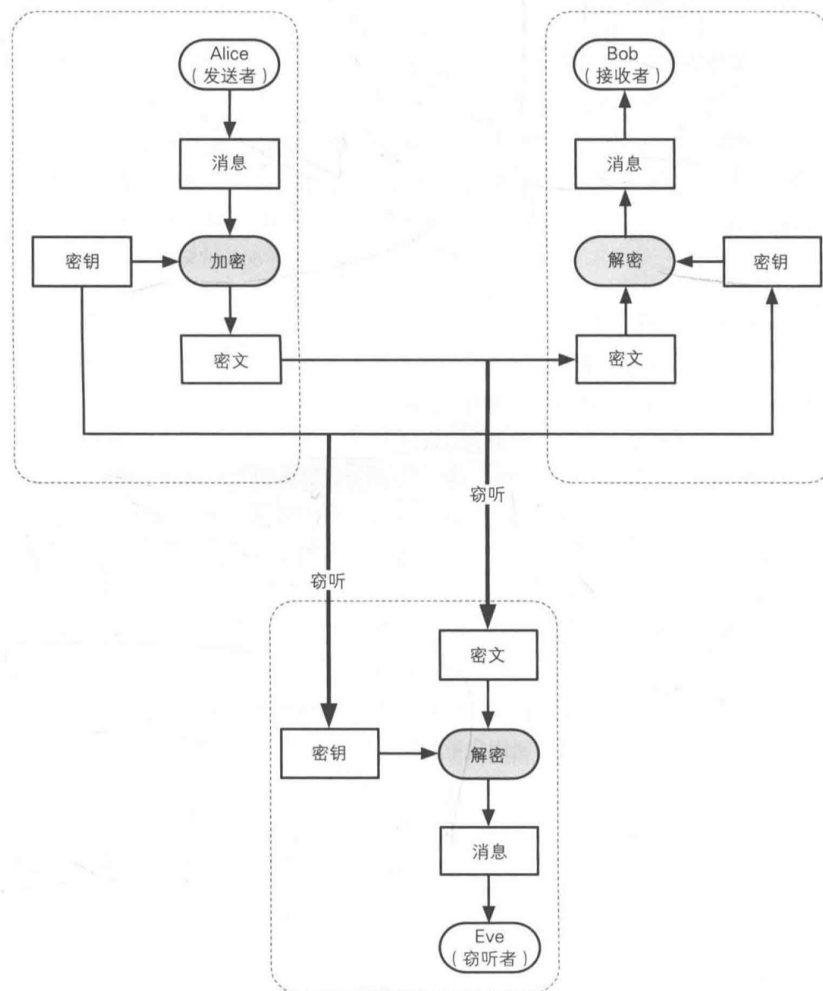
顾辰

guchen@hfut.edu.cn



3.1 密钥配送问题

- 如何安全地将密钥分发给需要解密的人？



3.1 密钥配送问题

□ 方案1：事先共享

➤ 依旧难以解决

➤ 密钥数量巨大

例如公司1000个员工需要彼此加密通信，密钥数量为：
 $1000 * 999 / 2 = 499500$

3.1 密钥配送问题

□ 方案2：密钥分配中心

1. Alice 向密钥分配中心发出希望与 Bob 进行通信的请求。
2. 密钥分配中心通过伪随机数生成器生成一个会话密钥，这个密钥是供 Alice 与 Bob 在本次通信中使用的临时密钥。
3. 密钥分配中心从数据库中取出 Alice 的密钥和 Bob 的密钥。
4. 密钥分配中心用 Alice 的密钥对会话密钥进行加密，并发送给 Alice。
5. 密钥分配中心用 Bob 的密钥对会话密钥进行加密，并发送给 Bob。
6. Alice 对来自密钥分配中心的会话密钥（已使用 Alice 的密钥加密）进行解密，得到会话密钥。

3.1 密钥配送问题

□ 方案2：密钥分配中心

7. Alice 用会话密钥对邮件进行加密，并将邮件发送给 Bob。
8. Bob对来自密钥分配中心的会话密钥（已使用 Bob 的密钥加密）进行解密，得到会话密钥。
9. Bob 用会话密钥对来自 Alice 的密文进行解密。
10. Alice 和Bob 删除会话密钥。

缺点：

复杂

密钥中心的脆弱性

3.1 密钥配送问题

□ 方案3: Diffie-Hellman密钥交换

- 进行加密通信的双方需要交换一些信息，而这些信息即便被窃听者 Eve 窃听到也没有问题。
- 根据所交换的信息，双方可以各自生成相同的密钥，而窃听者 Eve 却无法生成相同的密钥。
- Eve 虽然能够窃听到双方所交换的信息，但却无法根据这些信息生成和双方相同的密钥。

3.1 密钥配送问题

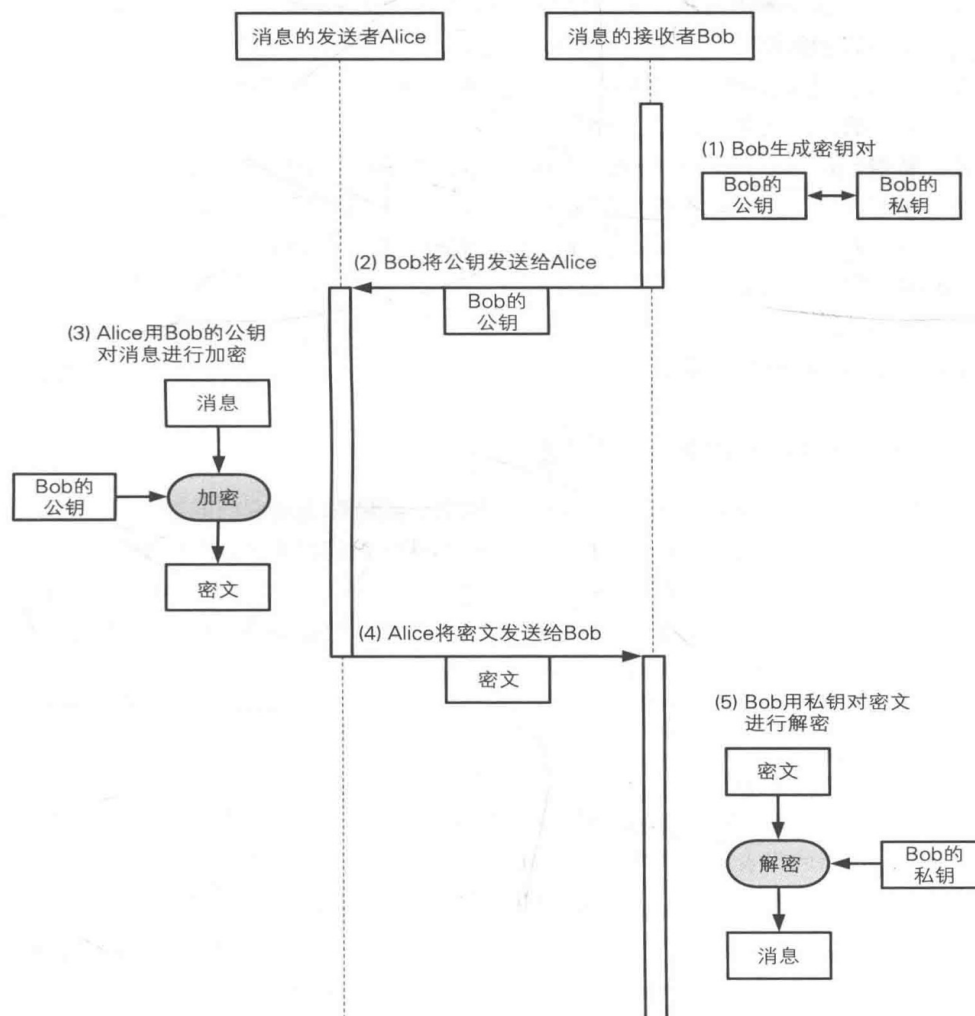
□ 方案4：通过公钥密码

- 在对称密码中，加密密钥和解密密钥是相同的，但公钥密码中，加密密钥和解密密钥却是不同的。
- 只要拥有加密密钥，任何人都可以进行加密，但没有解密密钥是无法解密的。因此，公钥密码的一个重要性质，就是只有拥有解密密钥的人才能够进行解密。
- 接收者事先将加密密钥发送给发送者，这个加密密钥即便被窃听者获取也没有问题。发送者使用加密密钥对通信内容进行加密并发送给接收者，而只有拥有解密密钥的人（即接收者本人）才能够进行解密。

3.2 公钥密码

- ❑ 公钥密码中，加密密钥一般是公开的。正是由于加密密钥可以任意公开，因此该密钥被称为公钥 (public key)。
- ❑ 解密密钥是绝对不能公开的，这个密钥只能由自己来使用，因此称为私钥 (private key)。
- ❑ 由公钥进行加密的密文，必须使用与该公钥配对的私钥才能够解密。密钥对中的两个密钥之间具有非常密切的数学上的关系，因此公钥和私钥是不能分别单独生成。
- ❑ 使用者需要生成一个包括公钥和私钥的密钥对。其中公钥会被发送给别人，而私钥则仅供自己使用。

3.2 公钥密码



3.2 RSA

- RSA 是一种公钥密码算法，它的名字是由它的三位开发者，即 Ron Rivest, Adi Shamir 和 Leonard Adleman 的姓氏的首字母组成的 (Rivest-Shamir-Adleman)。
- RSA加密：密文=明文 $E \bmod N$ (E, N的组合为公钥)
- RSA解密：明文=密文 $D \bmod N$ (D, N的组合为私钥)

3.2 RSA

□ RSA生成密钥对

➤ 求N

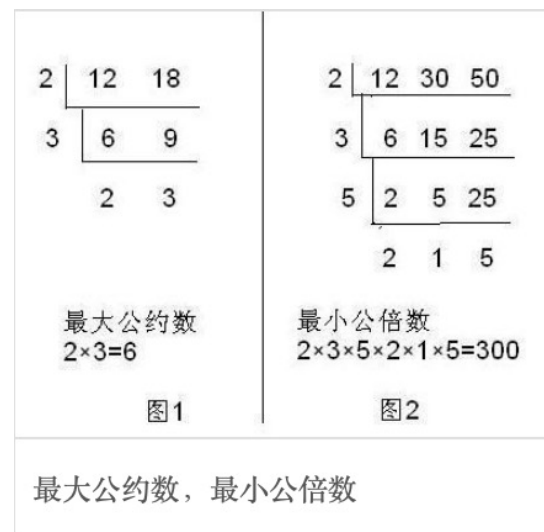
➤ 求L (L为中间数)

➤ 求E

➤ 求D

□ 求N: 随机生成两个大质数 p, q , 计算 $N = p * q$

□ 求L: L是 $(p-1)$ 和 $(q-1)$ 的最小公倍数 $L = \text{lcm}(p-1, q-1)$ 。



3.2 RSA

- 求E: E是一个比1大、比L小的数。此外, E和L的最大公约数 (greatest common divisor, gcd)必须为 1 (E和L互质)。
- 求D: $E * D \bmod L = 1$ ($1 < D < L$)

(1) 求 N	(3) 求 E
用伪随机数生成器求 p 和 q , p 和 q 都是质数 $N = p \times q$	$1 < E < L$ $\gcd(E, L) = 1$; E 和 L 的最大公约数为 1 (E 和 L 互质)
(2) 求 L	(4) 求 D
$L = \text{lcm}(p-1, q-1)$; L 是 $p-1$ 和 $q-1$ 的最小公倍数	$1 < D < L$ $E \times D \bmod L = 1$

3.2 RSA举例

- 求N: 选择质数 $p=17, q=19$, 则 $N=17*19=323$
- 求L: $L = \text{lcm}(p-1, q-1) = \text{lcm}(16, 18) = 144$
- 求E: 选择5 (不止一个)。则 $(E=5, N=323)$ 为公钥
- 求D: 满足 $E * D \bmod L = 1$ ($1 < D < L$), $D = 29$ 。则 $(D=29, N=323)$ 为私钥
- 公钥可以公开, 私钥不能公开。

3.2 RSA举例

□ 加密：密文=明文^E mod N =
123⁵ mod 323=225

□ 解密：明文=密文^D mod N
= 225²⁹ mod 323 = 123

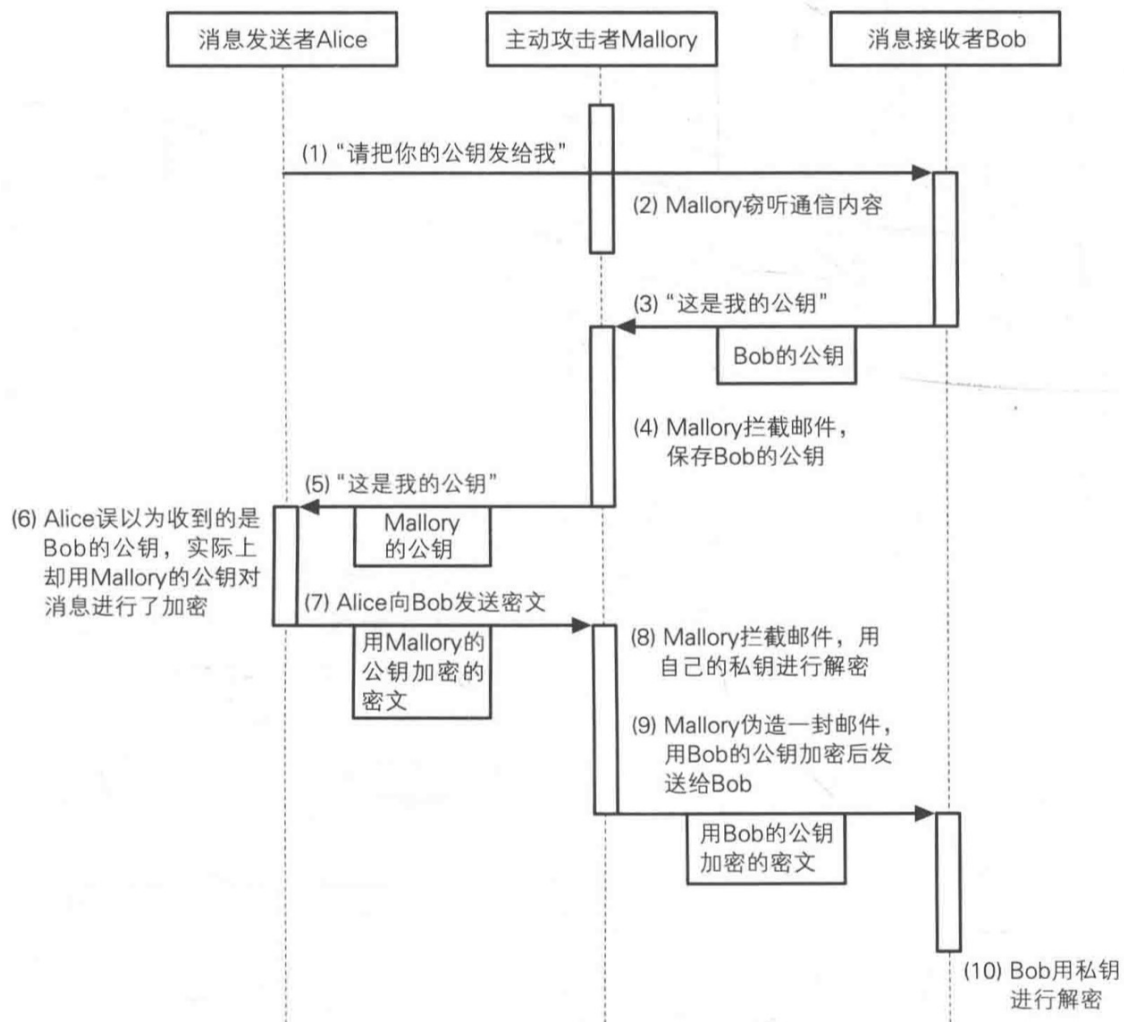
【密码破译者知道的信息】

- 密文：可以通过窃听来获取
- 数 E 和 N ：公钥是公开的信息，因此密码破译者知道 E 和 N

【密码破译者不知道的信息】

- 明文：需要破译的内容
- 数 D ：私钥中至少 D 是不知道的信息
- 其他：密码破译者不知道生成密钥对时所使用的 p 、 q 和 L

3.2 RSA的中间人攻击



3.2 RSA的其他问题

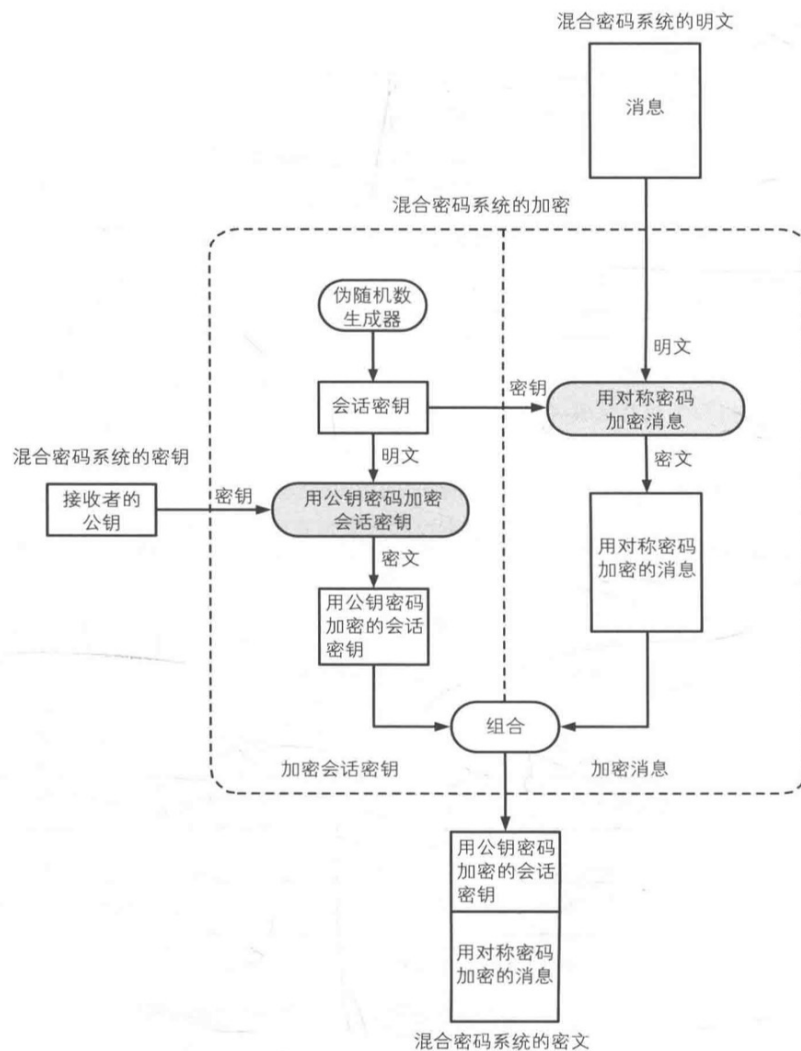
- ❑ 公钥密码是否比对称密码安全？
 - 不是。取决于密钥的长度。
- ❑ 密钥长度为256比特的对称密码 AES，与密钥长度为 1024 比特的公钥密码RSA相比，RSA 的安全性更高吗？
 - 不是。并不能简单对比密钥长度。

对称密码 AES	公钥密码 RSA
128	3072
192	7680
256	15360

3.2 其他问题

- 已经有了公钥密码，今后对称密码会消失吗？
 - 不会。一般来说，在采用具备同等机密性的密钥长度的情况下，公钥密码的处理速度只有对称密码的几百分之一。因此，公钥密码并不适合用来对很长的消息内容进行加密。
- 随着越来越多的人在不断地生成 RSA 的密钥对，质数会不会被用光呢？
 - 不用担心。512 比特能够容纳的质数的数量大约为 10 的 150 次方，这个数量比整个宇宙中原子的数量还要多。

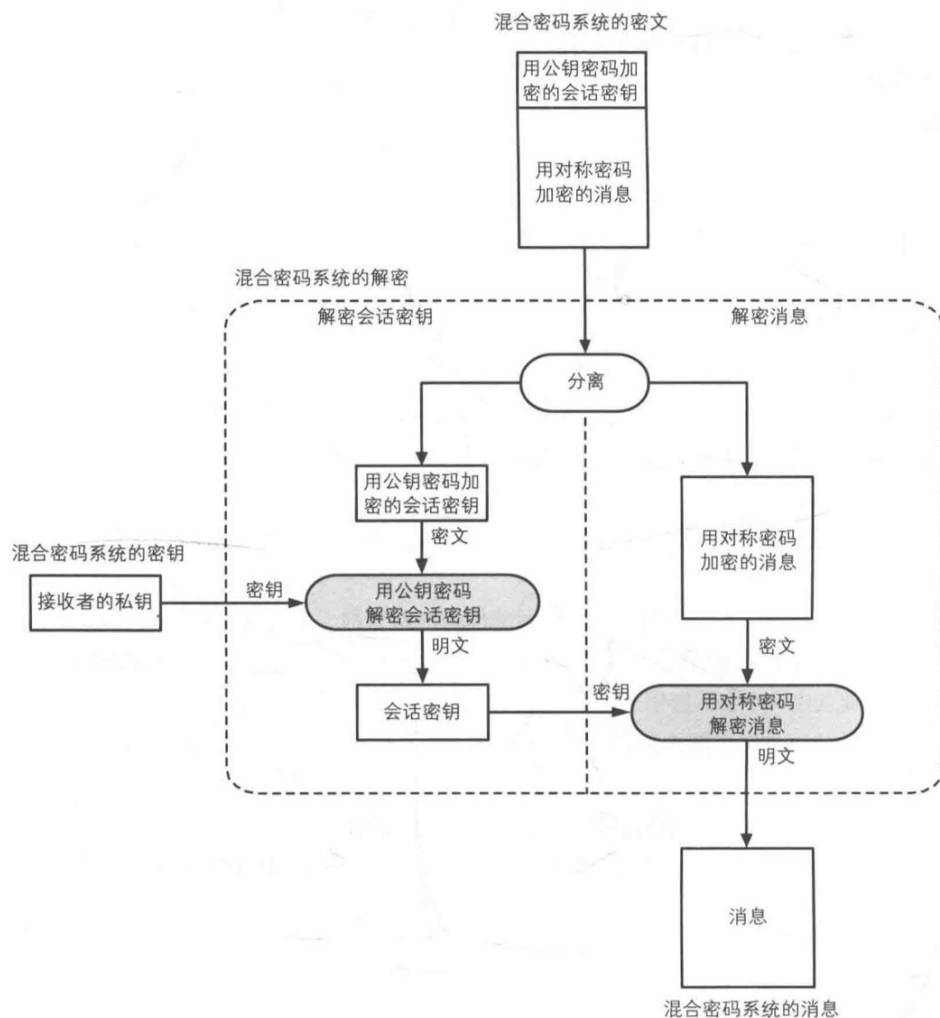
3.3 混合密码系统



- 对称密钥提高速度，非对称密钥保护会话密钥。
- 会话密钥是对称密码的密钥，同时也是公钥密码的明文。

3.3 混合密码系统

- 混合密码需要考虑伪随机数生成器，密钥长度的平衡。



3.4 随机数

□ 随机数是干什么用的？

- 生成密钥用于对称密码和消息认证码。
- 生成密钥对用于公钥密码和数字签名。

□ 随机数的性质

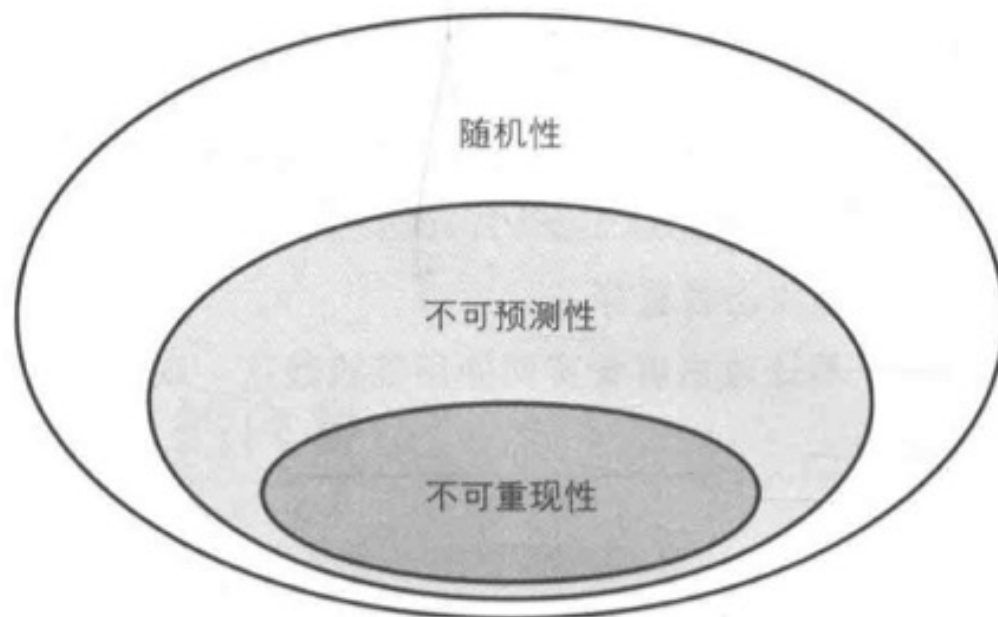
- 随机性：不存在统计学偏差，是完全杂乱的数列
- 不可预测性：不能从过去的数列推测出下一个出现的数
- 不可重现性：除非将数列本身保存下来，否则不能重现相同的数列

越往下，越严格

3.4 随机数

	随机性	不可预测性	不可重现性	
弱伪随机数	○	×	×	只具备随机性
强伪随机数	○	○	×	具备不可预测性
真随机数	○	○	○	具备不可重现性

↑ 不可用于密码技术
↓ 可用于密码技术

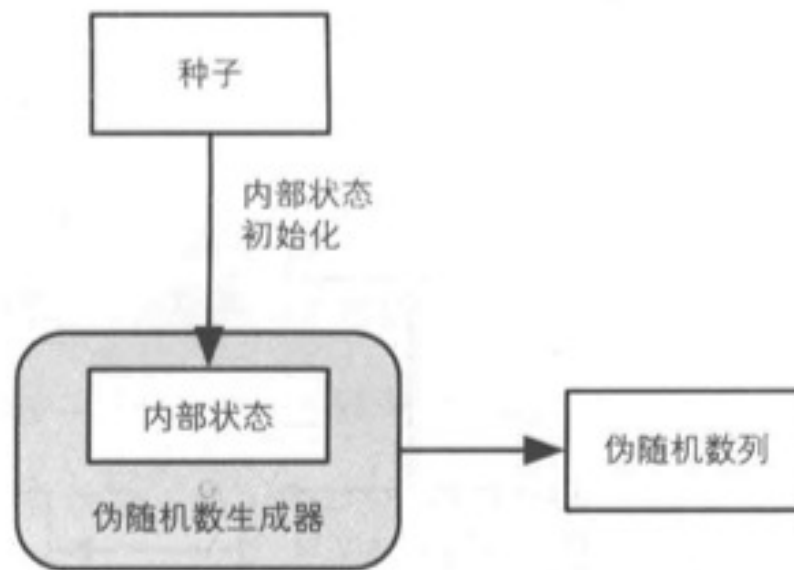


3.4 伪随机数生成器

- 随机数可以通过硬件来生成，也可以通过软件来生成。
- 通过硬件生成的随机数列，是根据传感器收集的热量、声音的变化等事实上无法预测和重现的自然现象信息来生成的。像这样的硬件设备就称为**随机数生成器** (Random Number Generator, RNG)
- 可以生成随机数的软件则称为伪随机数生成器 (Pseudo Random Number Generator, PRNG)。因为仅靠软件无法生成真随机数。

3.4 伪随机数生成器

- ❑ 伪随机数生成器具有“内部状态”，并根据外部输入的“种子”来生成伪随机数列
- ❑ 伪随机数生成器的**内部状态**，是指伪随机数生成器所管理的内存中的数值。当收到请求时，伪随机数生成器会根据内存中的数值（内部状态）进行计算，并将计算的结果作为伪随机数输出。随后伪随机数生成器会改变自己的内部状态
- ❑ 生成器需要被称为种子（seed）的信息。伪随机数的种子是用来对伪随机数生成器的内部状态进行初始化的



3.4 伪随机数生成器

- ❑ 杂乱的方法

- 复杂算法的周期短，难理解，无法判断是否具备不可预测性。

- ❑ 线性同余法

- ❑ 单向散列函数法

- ❑ 密码法

- ❑ ANSI x9.17

3.4 伪随机数生成器

❑ 线性同余法

- 线性同余法（linear congruential method）是一种使用很广泛的伪随机数生成器算法。然而，它并不能用于密码技术。
- 线性同余法就是将当前的伪随机数值乘以 A 再加上 C ，然后将除以 M 得到的余数作为下一个伪随机数。
- 在线性同余法中，最近一次生成的伪随机数的值就是内部状态，伪随机数的种子被用来对内部状态进行初始化。
- $R_{n+1} = (A * R_n + C) \bmod M$

3.4 伪随机数生成器

$$A = 3$$

$$C = 0$$

$$M = 7$$

然后将 6 作为伪随机数的种子，根据线性同余法，生成伪随机数列的过程如下。

$$\begin{aligned} R_0 &= (A \times \text{种子} + C) \bmod M \\ &= (3 \times 6 + 0) \bmod 7 \\ &= 18 \bmod 7 \\ &= 4 \end{aligned}$$

$$\begin{aligned} R_1 &= (A \times R_0 + C) \bmod M \\ &= (3 \times 4 + 0) \bmod 7 \\ &= 12 \bmod 7 \\ &= 5 \end{aligned}$$

$$\begin{aligned} R_2 &= (A \times R_1 + C) \bmod M \\ &= (3 \times 5 + 0) \bmod 7 \\ &= 15 \bmod 7 \\ &= 1 \end{aligned}$$

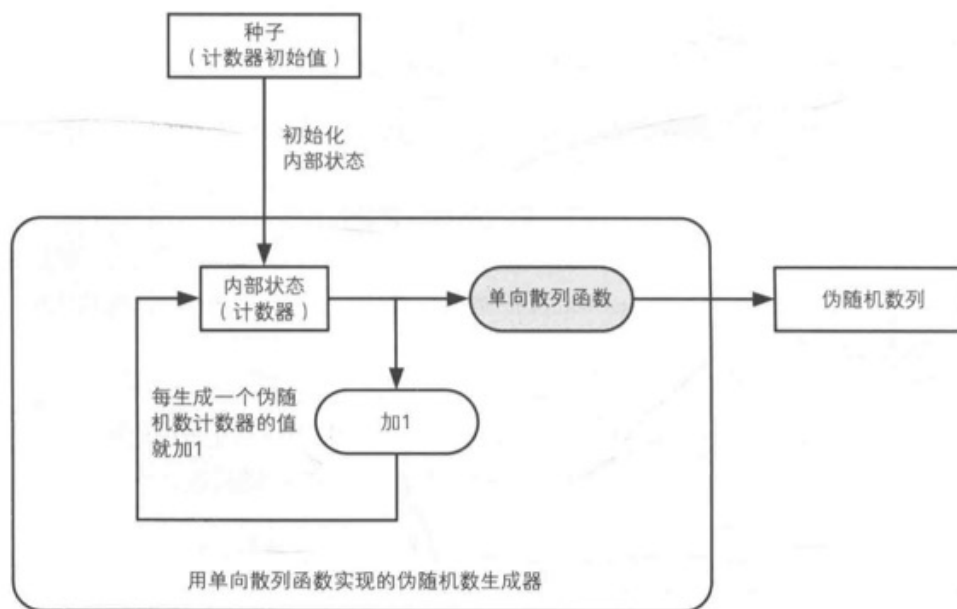
$$\begin{aligned} R_3 &= (A \times R_2 + C) \bmod M \\ &= (3 \times 1 + 0) \bmod 7 \\ &= 3 \bmod 7 \\ &= 3 \end{aligned}$$

3.4 伪随机数生成器

- 以此类推，我们可以得到4、5、1、3、2、6、4、5、1、3、2、6...这样的伪随机数列。这里数列是以4、5、1、3、2、6的顺序不断循环的，因此周期为6。
- 由于伪随机数是除以M得到的余数，因此其范围必定为 $0 \sim M-1$ ，而且根据 A、C和M的值，最终只能生成上述范围中的一部分值（因此周期会缩短）。例如，当 $A=6$ 、 $C=0$ 、 $M=7$ ，且种子为6时，所得到的伪随机数列为1、6、1、6、1、6...即周期为2。
- 线性同余法不具备不可预测性，因此不可以将线性同余法用于密码技术。
- 很多伪随机数生成器的库函数（library function）都是采用线性同余法编写的。例如C语言的库函数rand，以及Java 的 java.util.Random 类等，都采用了线性同余法。

3.4 单项散列函数法

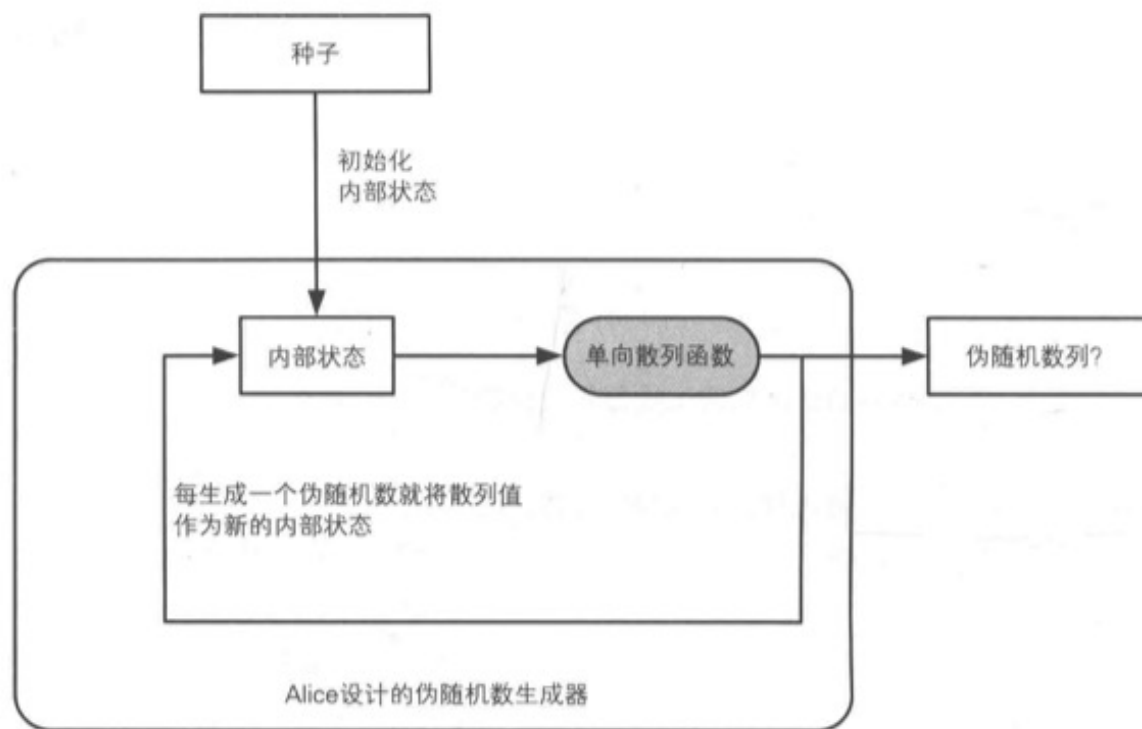
- 使用单向散列函数（如 SHA-1）可以编写出能够生成具备不可预测性的伪随机数列（即强伪随机数）的伪随机数生成器。



- 问题：攻击者能否预测出下一个伪随机数呢？

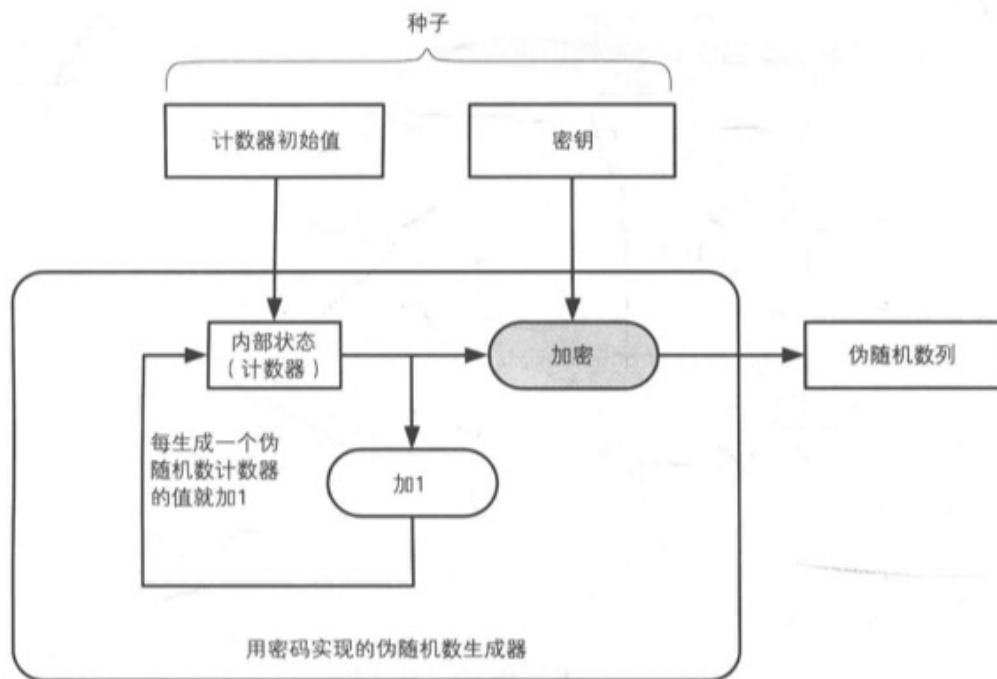
3.4 单项散列函数法

□ 如下设计的伪随机生成器是否可行？



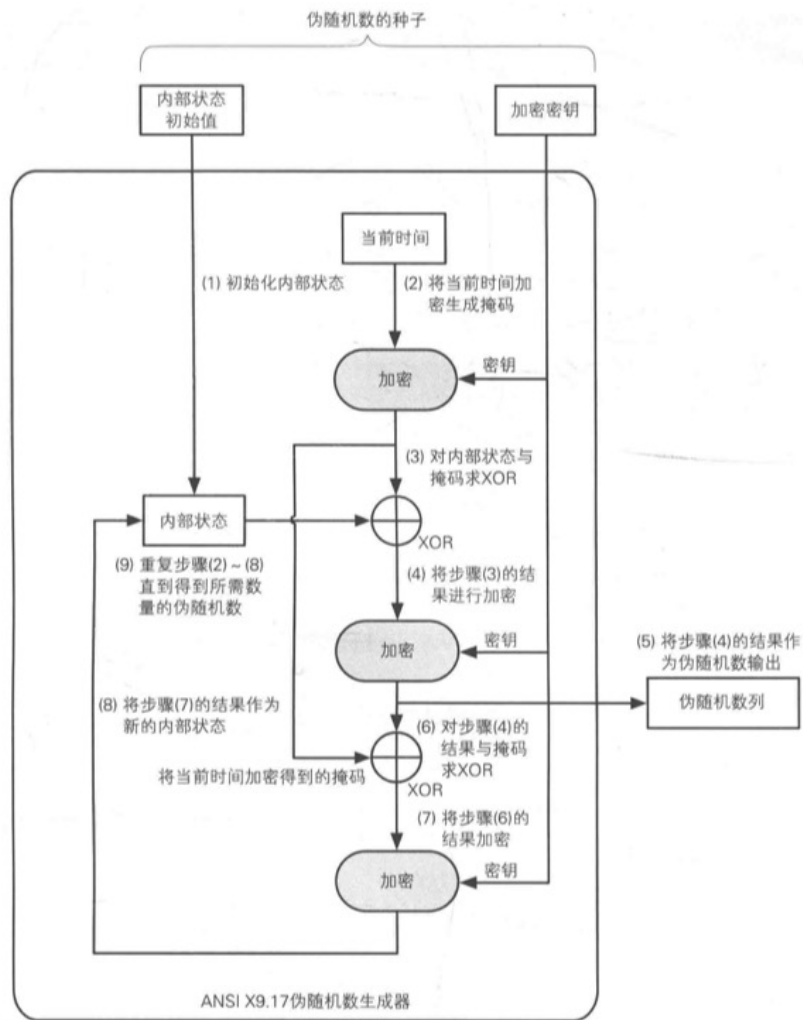
3.4 密码法

- 我们可以使用密码来编写能够生成强伪随机数的伪随机数生成器。既可以使用**AES** 等对称密码，也可以使用**RSA** 等公钥密码。



- 密码的机密性是支撑伪随机数生成器不可预测性的基础。

3.4 ANSI X9.17



3.4 对伪随机数生成器的攻击

- 由于伪随机数生成器承担了生成密钥的重任，因此它经常成为攻击的对象。
- 对种子进行攻击
 - 伪随机数的种子和密钥同等重要。如果攻击者知道了伪随机数的种子，那么就能够知道随机数生成器所生成的全部随机数列。因此，随机数的种子不可以被攻击者知道。要避免种子被攻击者知道，我们需要使用具备不可重现性的真随机数作为种子。
 - linux系统中/dev/random文件是收集背景噪声产生的真随机数
- 对随机数池进行攻击
 - 随机数会事先在一个名为随机数池（random pool）的文件中积累随机比特序列。当密码软件需要伪随机数的种子时，可以从这个随机数池中取出所需长度的随机比特序列来使用。随机数池的内容不可以被攻击者知道，否则伪随机数的种子就有可能被预测出来。